

Submitted By: Sparsh Singh

Erollment No. : 2023BTCSE030

Submitted To : Dr. Basant Agrawal

Data: 28 April, 2026

Heart Disease Prediction System

➤ PROJECT OVERVIEW

This project implements a predictive model to assess the risk of heart disease

in patients using machine learning techniques. The system combines:

- Data analysis and model training (Jupyter Notebook)
- Machine learning model deployment (KNN classifier)
- Interactive web interface (Streamlit application)

➤ DATASET DETAILS

Dataset Name: heart.csv

Total Features: 12

Target Variable: HeartDisease (binary: 0 = No disease, 1 = Disease present)

Feature Descriptions:

1. Age - Patient's age (years)
2. Sex - Biological sex (M = Male, F = Female)

3. ChestPainType - Type of chest pain experienced
(ATA = Atypical Angina, NAP = Non-anginal Pain,
TA = Typical Angina, ASY = Asymptomatic)
4. RestingBP - Resting blood pressure (mm Hg)
5. Cholesterol - Serum cholesterol level (mg/dL)
6. FastingBS - Fasting blood sugar > 120 mg/dL (0 = No, 1 = Yes)
7. RestingECG - Resting electrocardiogram results
(Normal, ST = ST-T wave abnormality,
LVH = Left ventricular hypertrophy)
8. MaxHR - Maximum heart rate achieved (bpm)
9. ExerciseAngina - Exercise-induced angina (Y = Yes, N = No)
10. Oldpeak - ST depression induced by exercise (relative to rest)
11. ST_Slope - Slope of the ST segment during exercise
(Up = Upsloping, Flat = Flat, Down = Downsloping)
12. HeartDisease - Target variable (0 = Low risk, 1 = High risk)

Data Quality:

- No duplicate records expected after preprocessing
- Categorical features are encoded (one-hot encoding)
- Numerical features are scaled for model compatibility
- Missing values handled during preprocessing

➤ **MODEL DEVELOPMENT SUMMARY**

Training Approach:

- Data split: Training (80%) and Testing (20%)
- Feature scaling: StandardScaler for normalization
- Categorical encoding: One-hot encoding for categorical variables

Algorithms Tested:

The notebook includes evaluation of multiple machine learning algorithms:

1. K-Nearest Neighbors (KNN) - Selected for deployment
2. Decision Tree Classifier
3. Gaussian Naive Bayes
4. Support Vector Classifier (SVC)

Model Selection:

- KNN model was selected as the primary algorithm for deployment
- Provides good balance between accuracy and interpretability
- Suitable for real-time predictions in production environment

Performance Metrics:

- Accuracy, F1 Score, and Precision measured on test set
- Models evaluated using cross-validation techniques
- Visualization of results through confusion matrices and ROC curves

Model Artifacts:

- Trained KNN model serialized as: knn_heart_model.pkl
- Feature scaler serialized as: heart_scaler.pkl
- Expected column names saved as: heart_columns.pkl

➤ **STREAMLIT APPLICATION**

Application Name: Heart Stroke Prediction

Purpose: Interactive risk assessment for heart disease

User Input Fields:

1. Age (slider): 18-100 years
2. Sex (dropdown): Male (M) or Female (F)
3. Chest Pain Type (dropdown): ATA, NAP, TA, ASY

4. Resting Blood Pressure (input): 80-200 mm Hg
5. Cholesterol (input): 100-600 mg/dL
6. Fasting Blood Sugar (dropdown): 0 or 1
7. Resting ECG (dropdown): Normal, ST, LVH
8. Max Heart Rate (slider): 60-220 bpm
9. Exercise-Induced Angina (dropdown): Y or N
10. Oldpeak - ST Depression (slider): 0.0-6.0
11. ST Slope (dropdown): Up, Flat, Down

Prediction Output:

The application displays:

- "High Risk of Heart Disease" (if prediction = 1) - displayed in red error box
- "Low Risk of Heart Disease" (if prediction = 0) - displayed in green success box

Data Processing Pipeline:

- Raw input collection from user interface
- Feature dictionary creation with proper naming conventions
- One-hot encoding of categorical variables
- Feature scaling using pre-trained StandardScaler
- Model prediction using trained KNN model

- Risk classification and result display

Technical Stack:

- Framework: Streamlit (Python web framework)
- Libraries: pandas, scikit-learn, joblib
- Deployment: Local or cloud-based Streamlit server

➤ **KEY COMPONENTS**

Data Preprocessing (HeartdiseaseFinal.ipynb):

- Data loading and initial exploration
- Handling missing values and duplicates
- Statistical analysis and feature distributions
- Exploratory data analysis with visualizations
- Categorical variable encoding
- Feature scaling and normalization

Model Training:

- Feature-target separation (X, y)
- Train-test split
- Model instantiation and training
- Hyperparameter tuning (if applicable)

- Model evaluation and comparison
- Model serialization and export

Deployment (app.py):

- Model loading from pickle files
- User interface development with Streamlit
- Input validation and preprocessing
- Real-time predictions
- Result interpretation and display

➤ **SYSTEM REQUIREMENTS**

Software Dependencies:

- Python 3.7+
- streamlit
- pandas
- scikit-learn
- joblib

Installation:

pip install streamlit pandas scikit-learn joblib

Running the Application:

streamlit run app.py

The application will open in a web browser at:

<http://localhost:8501>

➤ **WORKFLOW SUMMARY**

1. Data Analysis Phase (Notebook):

- Load heart.csv dataset
- Explore data structure and statistics
- Visualize feature distributions
- Identify correlations and patterns

2. Model Development Phase (Notebook):

- Preprocess data (encoding, scaling)
- Train multiple algorithms
- Evaluate model performance
- Select best performing model (KNN)

3. Model Export Phase (Notebook):

- Save trained model

- Save scaler
- Export expected column names

4. Deployment Phase (app.py):

- Load serialized model components
- Create interactive Streamlit interface
- Accept user input
- Make predictions in real-time
- Display results to users

➤ **LIMITATIONS & NOTES**

Current Limitations:

- Model accuracy depends on input data quality
- Predictions should be used as supplementary tool only
- Not a substitute for professional medical diagnosis
- Model trained on specific dataset characteristics
- Performance may vary with different demographic distributions

Future Improvements:

- Implement ensemble methods for better accuracy

- Add model explainability features (SHAP values, LIME)
- Include confidence scores in predictions
- Add data validation with error handling
- Implement model retraining pipeline
- Add logging and monitoring capabilities
- Deploy to cloud platform for wider accessibility